

Zou_lab_control.neutral_atom 硬件接入教程

qCMOS / FPGA sequencer / frontend live plotting quickstart

从 virtual 离线闭环到真实双 PC 触发架构的第一版实验线路

2026-06-02

Contents

1	目标和边界	1
1.1	当前目标	1
1.2	当前承诺和暂时不承诺的部分	1
1.3	不要让 notebook 变成命令行	2
2	对象模型	3
2.1	三层边界	3
2.2	为什么 capture 属于 camera	3
2.3	为什么 readout 是 subsystem	4
2.4	standalone operations 的位置	4
2.5	result object 和 summary	5
3	时序和触发	6
3.1	PulseSequence 是唯一时序源	6
3.2	多帧 acquisition 的触发规则	6
3.3	preflight 的意义	7
3.4	Verilog 和 runtime edge table	7
4	真实硬件双 PC 架构	8
4.1	两台机器的职责	8
4.2	ManualSequencer 的用途	8
4.3	RemoteSequencer 的用途	9
4.4	Verilog/FPGA 电脑上跑什么	9
4.5	pulse-streamer 设计、资源和 latency	9
4.6	和 Confocal_GUI pulse API 的对应关系	11
4.7	Pulse GUI 的位置和启动方式	12
4.8	Pulse GUI 的 40 channel 使用方式	14
4.9	把 FPGA 烧成 pulse-streamer	16
4.10	硬件配置模板	19
4.11	device loader 的扩展原则	19
5	配置文件逐项解释	21
5.1	device graph 的解析方式	21

5.2	QCMOSCamera 配置	22
5.3	ManualSequencer 配置	22
5.4	RemoteSequencer 配置.	23
6	Acquisition 生命周期	24
6.1	完整顺序.	24
6.2	为什么 prepare 在 camera arm 前	24
6.3	为什么 wait done 在读完 frame 后	25
6.4	异常路径.	25
6.5	每一层该记录什么.	25
7	Readout 算法细节	26
7.1	site finding	26
7.2	ROI counts	26
7.3	threshold 和 fidelity	26
7.4	detection-time scan 的参考图像	27
8	真实机器迁移清单	28
8.1	阶段 0: 离线 smoke test 只跑 virtual tutorial	28
8.2	阶段 1: Verilog/FPGA 电脑启动 server.	28
8.3	阶段 2: control 电脑 real config dry-run	28
8.4	阶段 3: manual first-light.	29
8.5	阶段 4: remote service first-light	29
8.6	阶段 5: readout calibration	29
9	控制电脑 Jupyter 操作流程	30
9.1	检查配置和 API	30
9.2	连接真实 hardware session	30
9.3	拍 raw image	31
9.4	校准 sitemap	31
9.5	校准 thresholds	31
9.6	单次 detect	33
9.7	scan detection time	33
10	真实机器上的执行顺序.	35
10.1	first-light 手动触发	35
10.2	remote 自动触发	35
10.3	不要把数据采集写成用户线程	36
11	常见问题和排查	37
11.1	ModuleNotFoundError	37
11.2	capture、sitemap 和 detect 的显示区别	37
11.3	多帧真实采集少帧或 timeout	37
11.4	detection-time fidelity 随时间变长反而下降	38
11.5	PyQt 主线程和后台采集	38

12	下一步扩展计划	39
12.1	device 层	39
12.2	timing 层	39
12.3	readout 层	39
12.4	frontend 和 GUI 层	40

1

目标和边界

1.1 当前目标

`Zou_lab_control.neutral_atom` 当前提供一条真实硬件上最早会用到的实验闭环：

Experiment path

```
connect devices
-> configure imaging pulse sequence
-> capture raw camera image
-> calibrate sitemap
-> calibrate thresholds
-> detect occupancy
-> scan detection time and readout fidelity
```

这条线看起来短，但它决定了控制框架的关键边界：camera 是 device, sequencer 是 device, readout 是一个有机 subsystem, frontend 只负责画图和 live refresh, analysis 函数可以 standalone 处理已经保存的 array。以后换真实 qCMOS、FPGA、AOD rearrangement、GUI 面板或 database logging 时，实验逻辑仍应沿着这些边界扩展。

1.2 当前承诺和暂时不承诺的部分

当前版本承诺以下事情：

- `tutorials/neutral_atom_hardware_quickstart.ipynb` 是真实硬件 notebook：会连接 qCMOS 和 Verilog/FPGA 电脑，不再用 virtual device 冒充硬件。
- 同一个调用形状可以切到 `QCMOSCamera`、`ManualSequencer` 或 `RemoteSequencer`。
- `frames=N` 的 camera acquisition 会在 camera 边界统一检查 trigger 数量；单 shot imaging sequence 会自动 repeat 成 N 个 camera trigger。
- 所有画图使用 `Zou_lab_control.frontend`，包括 pulse sequence、raw image、sitemap、threshold histogram、detect image 和 detection-time scan。
- device 必须继承 `CameraDevice`、`SequencerDevice` 或 `TrapArrayDevice`，不靠 duck

typing 混过去。

当前版本暂时不承诺以下事情：

- 不把 address rearrangement、AOD waveform、full scheduler 一次性写完。
- 不把旧 `PythonCamDemo` 或旧 address-switch Python 接口作为 notebook 调用路径；真实硬件通过新的 `QCMOSCamera`、`RemoteSequencer` 和 `sequencer_server` 边界进入。
- 不包含最终实验版 FPGA runtime bitstream；Verilog/FPGA 电脑先通过新 server 的 command backend 调 Vivado Tcl 或实验室自定义 FPGA backend。

核心取舍

当前版本先覆盖第一条真实实验闭环，同时保留清晰的 device、timing、analysis 和 frontend 边界。后续增加 device、实验任务或 GUI 时，可以沿着这些接口扩展。

1.3 不要让 notebook 变成命令行

notebook 里每个 cell 应该是实验动作，而不是命令行脚本的平铺。推荐的交互形状是：

Notebook shape

```
exp = na.connect(
    "remote_template",
    sequencer={"host": "192.168.0.20", "port": 18861},
    open_devices=True,
)

exp.timing.configure_imaging(exposure=2e-3, load=True)
capture = exp.camera.capture()
sitemap = exp.readout.sitemap(frames=20, grid_shape=(5, 7))
threshold = exp.readout.thresholds(frames=80, site=0)
shot = exp.readout.detect()
scan = exp.readout.detection_time(times, shots=20)
```

这里 `exp` 是实验 session，`exp.camera` 是 device 本体，`exp.readout` 是 camera-readout subsystem，`exp.timing` 是 timing/preflight subsystem。用户层调用短，但语义没有混在一起。

2

对象模型

2.1 三层边界

最重要的结构是三层：

Boundary

```
frontend
  plot arrays / pulse sequence / images
  no hardware knowledge

neutral_atom session and subsystems
  connect device graph
  own current calibration and result history
  schedule experiment-level readout actions

devices and operations
  CameraDevice / SequencerDevice / TrapArrayDevice
  standalone array algorithms
  timing and Verilog helpers
```

`frontend` 不知道 camera，也不知道 atom。它只知道 array、histogram、pulse rows、live session 和 selector。反过来，`QCMOSCamera.acquire()` 不知道 threshold、site circles 或 fitting。readout subsystem 站在中间，把 camera image、pulse sequence、calibration 和 frontend plot 串成实验动作。

2.2 为什么 capture 属于 camera

`capture` 的物理语义是 camera 采图，所以它是 `CameraDevice` 的方法：

Camera contract

```
class CameraDevice(BaseDevice):
    @property
    def exposure(self) -> float: ...
```

```
def configure(self, *, exposure=None, **kwargs) -> None: ...

def acquire(self, frames=1, *, sequence=None, sequencer=None,
    ↪ **kwargs) -> list[np.ndarray]: ...

def capture(self, *, frames=1, exposure=None, sequence=None,
    ↪ display=True, **kwargs) -> CaptureResult:
    ...
```

`acquire` 是底层动作: arm buffer、准备 sequencer、启动采集、读 frame、返回 image list。`capture` 是 notebook convenience: 调用 `acquire`, 把最后一帧画出来, 并返回 `CaptureResult`。它只显示 raw camera frame。site centers、ROI circles 和 occupancy overlays 属于 readout/calibration 结果, 因此只出现在 `sitemap`、`thresholds`、`detect` 等 readout 图里。

2.3 为什么 readout 是 subsystem

`sitemap`、`thresholds`、`detect` 和 `detection_time` 是一组有机动作。它们共享同一份 `TrapCalibration`:

- `sitemap` 给出 camera-space site centers、grid shape、ROI radius 和 ordering。
- `thresholds` 在这些 centers 上建立 per-site cut。
- `detect` 消费 centers 和 thresholds, 输出 occupancy。
- `detection-time scan` 在同一套 ROI/count model 上估计 readout fidelity。

因此它们放在 `ReadoutSubsystem`, 而不是散落成多个彼此看似独立的对象, 例如:

Anti-pattern

```
exp.sites.calibrate()
exp.threshold.calibrate()
exp.detector.detect()
```

拆得太碎会让依赖关系藏起来: `detect` 依赖 `sitemap` 和 `threshold`, `detection-time calibration` 依赖同一套 ROI 定义, 这些不是孤立工具函数。

2.4 standalone operations 的位置

虽然 readout 是一个 subsystem, 但核心算法仍然是 standalone:

Standalone analysis

```
sitemap = na.calibrate_sitemap_from_images(images, grid_shape=(5, 7))
threshold = na.calibrate_threshold_from_images(images,
    ↪ sitemap.calibration)
shot = na.detect_image(image, threshold.calibration)
```

这让离线重分析、batch processing 和未来 GUI 都能复用同一套 array algorithm。subsystem 只负责把当前 session 的 device/defaults/calibration/history 串起来, 不把算法写死在 session 里面。

2.5 result object 和 summary

每个实验动作返回 result object, 而不是只返回裸 array:

- `CaptureResult`: `images`、`image`、`sequence`、`plot`。
- `SitemapResult`: `calibration`、`centers`、`average_image`、`images`、`plot`。
- `ThresholdResult`: `counts`、`thresholds`、`selected_site`、`plot`。
- `DetectionResult`: `image`、`counts`、`occupied`、`occupied_indices`、`plot`。
- `DetectionTimeScanResult`: `times`、`fidelities`、`measurement` 和 `plot/data-fit handles`。

`summary()` 是轻量摘要, 面向 notebook 快速查看、GUI 状态栏和测试断言。它不是数据接口的替代品。真实分析时应该读 result object 上的 raw array 和 calibration。

3

时序和触发

3.1 PulseSequence 是唯一时序源

`PulseSequence` 用物理时间描述 channel、start、duration 和 value。它不直接等价于 Verilog，也不直接等价于 camera exposure；它是上层实验时序的 source of truth。当前 imaging sequence 大致包含：

- `trap`: 维持 trap/channel hold。
- `cooling`: 如果 `load=True`，在成像前给 load/cooling window。
- `probe`: camera integration 期间打开 probe。
- `qcm_trigger`: 在 probe 开始处给 qCMOS external trigger。

3.2 多帧 acquisition 的触发规则

真实 qCMOS external trigger 模式下，请求 N 帧意味着必须给 N 个 trigger。以前最容易出问题的地方是：notebook 里写 `frames=80`，但 sequence 只有一个 `qcm_trigger`。virtual camera 可能仍然能返回 80 张图，真实 camera 却会等不到后续 frame，最后 timeout 或得到不完整数据。

现在规则放在 camera acquisition 边界：

Trigger normalization

```
runtime_sequence = sequence_for_frame_count(sequence, frames)

if trigger_count(sequence) == frames:
    use sequence directly
elif trigger_count(sequence) == 1 and frames > 1:
    use sequence.repeated(frames)
else:
    raise before camera arm
```

这个位置很关键。它不属于 readout，不属于 notebook，也不属于 frontend。因为任何 camera acquisition 都必须满足这个规则，不管它来自 `capture`、`sitemap`、`thresholds` 还是 `detection_time`。

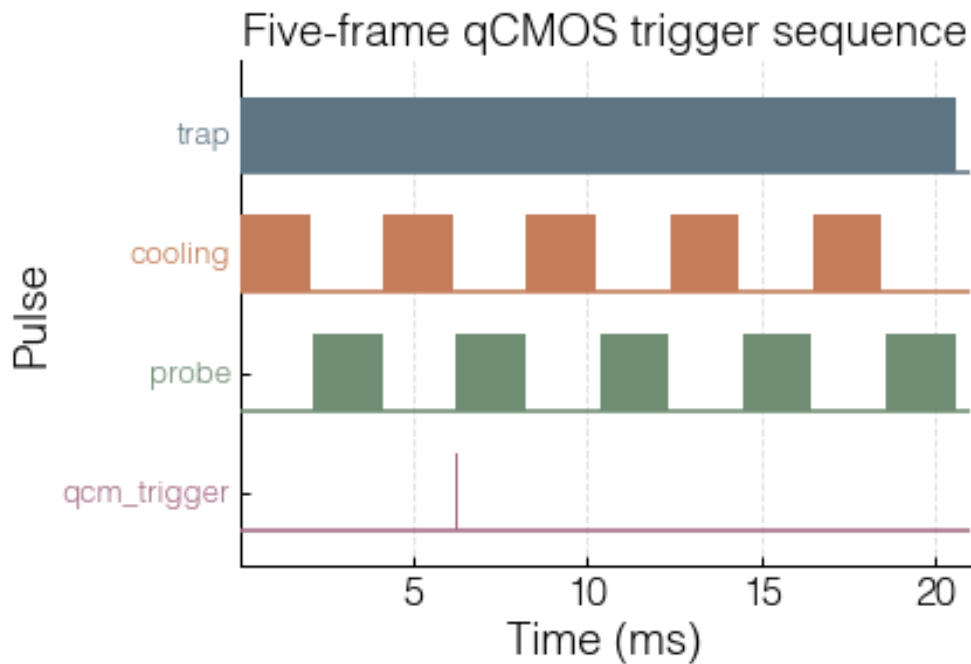


Figure 3.1: 同一个单 shot imaging template 被 repeat 成五个 qCMOS trigger。baseline 和 pulse block 使用同一颜色，便于检查 channel 对应关系。

3.3 preflight 的意义

`exp.timing.preflight()` 在真实 shot 前检查：

- sequence 中的 channel 是否在 sequencer channel list 里。
- pulse 是否重叠，duration 是否能落到 clock tick。
- Verilog/edge table 是否能生成。
- 当前 device snapshot 是否符合预期。

preflight 不是安全联锁，但它能提前抓住 “channel 名写错” “trigger 少了” “clock tick 太短” “配置文件不是当前硬件” 等低级错误。

3.4 Verilog 和 runtime edge table

当前代码里有两条 sequencer 路径：

- **VerilogSequencer**: 用 **PulseSequence** 生成 Verilog bundle，适合离线 Vivado 检查或静态 first-light。
- **RuntimeSequencer/RemoteSequencer**: 把 **PulseSequence** 编译成 ticks/masks edge table，适合最终 control PC 到 FPGA PC 的 runtime 协议。

两者的源头都是同一个 **PulseSequence**。这点很重要：不要让 notebook 维护一套 Python timing，又让 Verilog 另有一套手写 timing。否则 calibration 和真实 hardware shot 很快会分叉。

4

真实硬件双 PC 架构

4.1 两台机器的职责

推荐的长期结构是：

Two-PC architecture

Control PC

```
Jupyter / future GUI  
NeutralAtomSession  
QCMOSCamera DCAM handle  
RemoteSequencer client
```

FPGA/Vivado PC

```
SequencerService  
Vivado/FPGA runtime backend  
prepare(edge table)  
fire()  
wait_done()
```

网络/RPyC 只负责上传 sequence 和发 start 命令，不负责每个 pulse 的微秒级 timing。真正的 timing 必须由 FPGA clock 保证。这样网络 jitter 不会变成实验 jitter。

4.2 ManualSequencer 的用途

ManualSequencer 是 first-light 工具。它做的事情很少：

- 在 `prepare` 阶段验证 sequence 和 trigger 数量。
- 在 `fire` 阶段打印提示：camera 已经 arm，需要启动 FPGA/manual trigger。
- 不假装自己知道硬件已经真的完成，只返回一个保守的 snapshot state。

它适合第一次确认 qCMOS external trigger、电缆、TTL polarity、ROI 和 exposure。它不适合自动 scan，因为自动 scan 需要可编程、可等待、可 abort 的 sequencer。

4.3 RemoteSequencer 的用途

`RemoteSequencer` 是最终控制 PC 上的 device adapter。它通过 RPyC 连接 FPGA PC 上的 `SequencerService`，公开同一个 `SequencerDevice` contract:

Remote sequencer contract

```
sequencer.prepare(sequence)
camera arm
sequencer.fire(sequence)
camera wait/read frames
sequencer.wait_done(timeout)
```

`connect_on_init=False` 让 device graph 可以先构造和校验而不连接网络。需要在连接阶段直接打开硬件时，用 `na.connect(..., open_devices=True)` 或 `na.load_devices(..., open_devices=True)`，由 device loader 统一处理 open 顺序和失败回滚。

4.4 Verilog/FPGA 电脑上跑什么

Verilog/FPGA 电脑负责启动 sequencer server。推荐先打开 FPGA server notebook:

Notebook

```
tutorials/neutral_atom_fpga_server.ipynb
```

默认路径是新的 `fpga_pulse_streamer` backend: 固定烧一个 runtime-programmable pulse-streamer bitstream, control PC 每次只上传 `PulseSequence`; FPGA PC 把 sequence 编译成 `RuntimeSequenceProgram` 的 ticks/masks edge table, 再通过 VIO 写进 FPGA RAM。这样 readout time、multi-frame trigger、cooling/probe/trap pulse 都来自同一张 edge table, 不再把 sequence 压扁成旧 `pulse_lasting/cycle_counts` 两个参数。

4.5 pulse-streamer 设计、资源和 latency

`zlc_pulse_streamer` 的核心思想是 edge table, 而不是逐条 pulse 解释。Python 先把 `PulseSequence` 展开成按时间排序的边沿表:

Runtime program

```
tick[i] = integer FPGA clock tick
mask[i] = complete output state at tick[i]
```

`mask` 是所有输出 channel 的完整状态。比如 40 路输出时, `mask` 是 40 bit: bit 0 对应 `channels[0]`, bit 1 对应 `channels[1]`, 以此类推。多个 channel 如果在同一时刻改变, 它们会合并成同一条 edge; FPGA 到时刻后一次性更新整个 `state_mask`, 因此 qCMOS trigger、probe、trap、cooling 和其它 TTL channel 都来自同一个时钟域。

生成的 Verilog core 内部主要有:

Core state

```
tick_mem[MAX_EDGES] // TICK_WIDTH bits per edge
mask_mem[MAX_EDGES] // CHANNEL_COUNT bits per edge
state_mask           // current output register
time_count           // counter while running
```

```
edge_index      // next edge to apply
active_count    // uploaded edge count
```

`prepare` 阶段只写 RAM，不出光、不触发相机。当前 VIO backend 会先让 `reset=1`，写入 `prog_count`，然后对每个 edge 写 `prog_addr/prog_tick/prog_mask`，再把 `prog_we` 拉高/拉低打一拍。写完后释放 `reset`。这个动作可以很慢，因为它走 Vivado、hw server 和 JTAG；但它发生在 camera arm 之前，不参与 shot 内部 timing。

`fire` 阶段只负责给 `zlc_start` 一个边沿。qCMOS acquisition 的顺序是：先 `prepare(sequence)`，然后 camera `cap_start` 进入等待 external trigger 状态，最后 `fire()`。因此 Python/Vivado/JTAG 的启动延迟只决定“这一发 shot 什么时候开始”，不决定 shot 内每个 pulse 的相对时间。start edge 到达 FPGA 以后，所有 pulse edge 都由 FPGA clock 推进。

在 `CLOCK_HZ=100000000` 时，一个 tick 是 10 ns。Python 侧把物理时间换成 tick 使用 `round(time * clock_hz)`，所以单个 edge 的量化误差约为半个 tick，即 5 ns 量级。FPGA core 看到 `start` 后有一个固定 pipeline offset：约一拍同步、一拍检测 start、一拍应用 `tick=0` 的第一条 mask；100 MHz 下约 30 ns。这个 offset 是固定的，可以并入 qCMOS trigger channel 的定义；edge 之间的相对 timing 仍然在 10 ns tick grid 上。

资源估算时，真正占空间的是 edge table。40 路、32 bit tick 时，每条 edge 约为：

Edge size

```
32 bit tick + 40 bit mask = 72 bit per edge
```

对应的 RAM 量级是：

40-channel edge-table estimate

```
MAX_EDGES=1024  -> 73,728 bit  ~=  9 KiB
MAX_EDGES=4096  -> 294,912 bit ~= 36 KiB
MAX_EDGES=16384 -> 1.18 Mbit  ~= 144 KiB
```

当前 first-light HDL 为了让 `tick_mem[edge_index]` 和 `mask_mem[edge_index]` 在同一拍被简单读取，显式使用 Vivado distributed RAM，也就是 LUTRAM。对 40 路、1024 edges 来说，大约是 72k bit 量级，约等于一千多个 64x1 LUTRAM 单元，再加少量 counter/register/VIO 逻辑；这对 Artix-7 35T 档位是合理的 first-light 规模。更大的 4096/16384-edge 表也可以作为方向，但那时要看 Vivado utilization 里的 LUT/SLICEM 占用；如果 sequence 真的长期需要大表，应把 edge table 改成 BRAM-friendly 的同步读 pipeline 或 AXI/BRAM 写接口，而不是继续无限放大 VIO/LUTRAM 版本。

本地历史 Vivado 工程使用的 FPGA part 是 `xc7a35tfgg484-2`，也就是 Artix-7 35T 这个资源档。这个档位有大约 33k logic cells、20k LUT、41k flip-flop、50 个 36 Kb block RAM 和 90 个 DSP slice。对 40 路、1024-edge first-light pulse-streamer 来说，LUT/FF/BRAM 都不是主要瓶颈。更实际的瓶颈是三件事：

- 是否有 40 个已经引出的、空闲的、同电平兼容的物理输出 pin。
- 输出到实验仪器时是否需要 buffer、level shifter 或 line driver。FPGA pin 不应直接当作耐插拔的实验室 BNC/TTL 输出。
- VIO 上传速度。当前 `prepare` 大约需要 `5 * edge_count` 次 `commit_hw_vio` 加少量常数项，适合 first deployment 和小表；如果要高重复率 scan 或很长 sequence，后续应换成 AXI、UART、SPI、Ethernet、JTAG-to-AXI 或 FIFO/BRAM 写接口。

因此推荐的第一版参数是 `CHANNEL_COUNT=40`、`MAX_EDGES=1024`、`TICK_WIDTH=32`、`CLOCK_HZ=100000000`。先综合、看 utilization 和 timing，再根据真实 sequence 的 edge 数决定是否升到 4096 或切换 BRAM pipeline。40 路输出本身只是一个 40 bit register 和 40 个 top-level output；工程工作量主要在 top wrapper、VIO probe 宽度和 XDC pin map。

4.6 和 Confocal_GUI pulse API 的对应关系

旧 `Confocal_GUIv2` 里的 `BasePulse` 把 pulse 存成 `data_matrix`：每一行是一个 duration，后面跟 8 个 channel 的 0/1 状态；`read_data()` 再把它变成每个 channel 的 time slice。这个设计有三个值得保留的思想：

- 用户层描述 pulse，而不是手写硬件寄存器。
- channel delay 和 repeat 在编译阶段展开，硬件只执行最终时序。
- 真正打开 pulse 前先进入 off/safe state，结束时能回到 final/safe state。

当前 `Zou_lab_control.neutral_atom` 采用同样的边界，但数据结构换成更适合相机触发和多 PC 控制的形式：

Concept map

```
Confocal_GUI BasePulse.data_matrix
  duration row + per-channel state
  delay_array / repeat_info
  off_pulse() -> on_pulse_core()

Zou_lab_control PulseTableState / PulseSequence
  GUI period cards + visible channel subset
  x_ns / delays / repeat bracket
  to_sequence()

SequencerDevice
  absolute physical start/duration
  sequence.delay(channel, dt) / sequence.repeated(frames)
  compile_runtime_program() -> ticks/masks edge table
  prepare(reset/upload) -> fire(start) -> wait_done()/safe_state()
```

`PulseTableState` 是给 GUI 和 notebook 共用的编辑模型。它保留旧 pulse GUI 的 period-card 思路：每一段有一个 duration，并给出这一段内所有可见 channel 的 0/1 状态；`x_ns`、每路 delay 和 repeat bracket 在编译阶段展开。它不拥有硬件，也不直接写 VIO。用户按 period 编辑完成后，调用 `state.to_sequence()` 得到 `PulseSequence`；再由 `RemoteSequencer.prepare/fire/wait_done` 或本地 `RuntimeSequencer` 执行。

`PulseSequence` 保留物理时间语义，`compile_runtime_program` 才把它投影到 FPGA clock tick。这样 notebook 和 GUI 里看到的是 `probe`、`qcm_trigger` 和 `trap` 的物理时间；FPGA 看到的是已经展开、排序并验证过的 `tick/mask` 表。旧 `.npz pulse_file` 不作为新硬件路径的输入格式；新 GUI 使用 `PulseTableState.save("pulse.json")` 保存 JSON，里面包含 channel list、可见 channel、period、delay、repeat 和 channel display label。需要 scan 某个 pulse width 时，可以在 Python 层改 `x_ns` 或重建 `PulseTableState/PulseSequence`，然后重新 `prepare`。

所有 pulse 数字最终都必须是 minimal time 的整数倍。编辑模型里的 minimal time 是 `PulseTableState.time_step_ns`；连接 100 MHz FPGA server 时，GUI 默认把它设成 10 ns。API 也可以显式写 `time_step_ns=10`。`state.total_duration_steps(...)` 返回整数 step

数, `state.compile(clock_hz=...)` 会按 $1e9/\text{clock_hz}$ 再检查一次 FPGA tick。直接使用 `PulseSequence` 的用户也会在 `sequence.validate(clock_hz=...)` 和 server prepare 阶段被检查; 如果某个 start/stop 不在 clock grid 上, 会在上传前报错, 而不是 silently round。

`x_ns` 的使用方式接近旧 pulse GUI 里的 `x`, 但它留在 Python/GUI 前端层。duration 或 delay 选择 `str (ns)` 时可以写 `x` 表达式, 例如 `x`、`1000-x` 或 `2*x+50`。扫某个 duration 时, 不需要重写整张表:

Python

```
state = na.PulseTableState(
    channels=["trap", "probe", "qcm_trigger"],
    time_step_ns=10, # 100 MHz FPGA clock -> 10 ns tick
    periods=[
        na.PulsePeriod(1000, (1, 0, 0), unit="ns", name="pre"),
        na.PulsePeriod("x", (1, 1, 1), unit="str (ns)", name="image"),
        na.PulsePeriod(1000, (0, 0, 0), unit="ns", name="idle"),
    ],
)

clock_hz = exp.devices.sequencer.clock_hz
clock_step_ns = 1e9 / clock_hz
for width_ns in [500, 1000, 2000, 4000]:
    seq = state.to_sequence(x_ns=width_ns, time_step_ns=clock_step_ns)
    program = state.compile(clock_hz=clock_hz,
        ↪ trigger_channels=["qcm_trigger"], x_ns=width_ns)
    exp.devices.sequencer.prepare(seq)
    exp.devices.sequencer.fire()
    exp.devices.sequencer.wait_done(timeout=1.0)
```

每个 `x_ns` 值都会生成一张新的 edge table, 所以自动 scan 的正常顺序是 `prepare(seq_for_this_x)`, camera arm, `fire()`, 然后 `wait_done()`。如果只是离线检查, 可以用 `state.with_x(width_ns)` 生成带不同默认 `x_ns` 的 copy, 或者用 `state.to_sequence(x_ns=...)` 临时覆盖。

安全状态也对应得很直接: 旧 pulse streamer 的 `off_pulse()` 会先写一个全低 pattern; 这里 `prepare` 会先拉高 `reset`, `validate_pulse_streamer_program` 要求最后一条 mask 为 0, `safe_state` 会把 `start/prog_we` 拉低并保持 `reset=1`。因此每个正常 sequence 的最后状态、abort 状态和下一次 prepare 的初态都是全低输出。

4.7 Pulse GUI 的位置和启动方式

Pulse GUI 是 `Zou_lab_control.frontend` 的前端控件, 不是新的硬件层。它只编辑 `PulseTableState`, 并在用户按 `Prepare/Fire/Wait/Safe` 时调用传入的 `SequencerDevice`。因此同一个 GUI 可以在控制电脑上连远程 FPGA server, 也可以在 Verilog/FPGA 电脑上通过 `127.0.0.1` 连接本机正在运行的 server。

最简单的 standalone 打开方式是在仓库根目录运行:

PowerShell

```
.\pulse_gui.bat
```

这个入口不读取 experiment config。默认创建 40 路本地 `RuntimeSequencer` 预览, 硬件 channel 名是 `ch00...ch39`, 不自动把前几路显示成 `trap/cooling/probe/qcm_trigger`; 如果需要这

些实验含义，请在 GUI 的 Name 右侧手动填写或加载保存过的 JSON。窗口固定为当前屏幕可用区域的 90%；如果 FPGA server 已经在本机运行，使用：

PowerShell

```
.\pulse_gui.bat --remote-host 127.0.0.1 --remote-port 18861
```

控制电脑连接 FPGA/Vivado 电脑时，把 127.0.0.1 换成 FPGA 电脑 IP。

在控制电脑上，如果已经有实验 session：

Python

```
import Zou_lab_control.frontend as zf
import Zou_lab_control.neutral_atom as na

exp = na.connect(
    "remote_template",
    sequencer={"host": "192.168.0.20", "port": 18861},
    open_devices=True,
)

gui = zf.show_pulse_gui(experiment=exp)
```

`show_pulse_gui(experiment=exp)` 会从 `exp.devices.sequencer.channels` 读取真实 channel list，并把按钮连到同一个 sequencer。按 Prepare 时，GUI 先把 period card 编译成 `PulseSequence`，再调用 `RemoteSequencer.prepare(sequence)`；按 Fire 时调用 `RemoteSequencer.fire()`；按 Wait 时调用 `wait_done(timeout=10)`；按 Safe 时调用 `set_safe_state()` 或 `abort()`。

如果 GUI 启动时传入了 sequencer，它会用 `sequencer.clock_hz` 自动设置左侧的 `step (ns)`。例如 100 MHz server 显示 10 ns。用户仍然可以手动改 `step (ns)` 做离线设计；但按 Prepare 时会按实际 sequencer clock 再检查一次，任何不在 FPGA tick 上的 duration、delay 或 `x_ns` 都会在上传前报错。

在 Verilog/FPGA 电脑上，如果 server 已经在另一个 terminal 或 notebook kernel 中运行，可以从第二个 Python 进程连接本机 server：

Python

```
import Zou_lab_control.frontend as zf
import Zou_lab_control.neutral_atom as na

channels = ["ch00", "ch01", "ch02", "ch03"]
sequencer = na.RemoteSequencer(
    host="127.0.0.1",
    port=18861,
    channels=channels,
    clock_hz=100_000_000,
    trigger_channels=["ch03"],
)

gui = zf.show_pulse_gui(channels=channels, sequencer=sequencer)
```

如果只是离线设计 pulse，不连接硬件，也可以不给 sequencer：

Python

```
state = na.PulseTableState(channels=["ch00", "ch01", "ch02", "ch03"])
gui = zf.show_pulse_gui(state=state)
sequence = gui.to_sequence()
sequence.validate(clock_hz=100_000_000,
↳ channels=state.channels).raise_if_failed()
```

这种模式下 GUI 可以保存/加载 JSON, 也可以导出 `PulseSequence` 给 notebook 继续画图或 preflight; `Prepare/Fire` 不会直接驱动硬件。

4.8 Pulse GUI 的 40 channel 使用方式

40 channel bitstream 的 channel list 由 server 启动参数和 `RemoteSequencer.channels` 共同定义。这个 list 是硬件 channel 名和 FPGA bit order, 例如 `ch00/ch01/...`, 不是 display label。GUI 默认不会把 40 路全部铺满屏幕, 而是先显示硬件顺序前 4 路; 其它 channel 保留在右下角的 hidden-channel 下拉框中。这样简单成像 pulse 不会被 40 个 checkbox 淹没。

period card 仍然沿用旧 pulse GUI 的横向时间轴: 每张 card 是一段 duration, 左右拖动可以重排 period。每张 card 标题显示当前位置, 例如 `Period 1/5`。右侧时间轴有独立的横向滚动条; channel name、delay 和 period checkbox 共用同一个纵向滚动区域, 因此 40 路展开时每个 channel 行仍然对齐。可见 channel 超过 16 路时, GUI 会自动使用 compact 尺寸, 方便临时 `Show All` 检查 40 路状态。

GUI 会根据屏幕自动缩放并固定窗口大小。如果当前显示器较小, 可以在启动时传入 `scale` 或 `window_ratio`, 例如 `show_pulse_gui(..., scale=0.82, window_ratio=0.90)`。`scale` 只影响控件尺寸; `window_ratio` 只影响窗口占屏比例; 二者都不改变 sequence 时间、channel 顺序或 FPGA bit mask。

常用操作如下:

- **Add Channel**: 从隐藏列表里把某一路按硬件 channel 顺序加入当前视图, 不改变其它 channel 的 pulse 状态。
- **Name**: 左侧固定显示硬件 channel, 例如 `ch00`; 右侧是可选 display label。GUI 不会自动猜 `ch00` 是什么设备。右侧 name 改动后, Delay 行、period checkbox 和 Preview y 轴会显示这个 label, 但保存和编译仍然使用左侧硬件 channel。
- **X**: 把当前 channel 在所有 period 中设为 off, 但不自动隐藏; display name 和 delay 会留在 state 中。
- **Hide Off**: 一次性隐藏所有 period 中全程为 off 的 channel, 并至少保留 4 路可见 channel。它只检查 period 是否为 on; delay 非零不会阻止隐藏。channel label 和 delay 会留在 state 中; 隐藏只是视图变化。
- **Show All**: 临时显示全部 channel, 用于检查 40 路 pin map 或做全局审查。
- **Add Period/Remove Period**: 增删 period card。每张 card 的 duration 可以用 `ns/us/ms/s`, 也可以用 `str (ns)` 写 x 表达式, 例如 `1000-x`。
- **Add Bracket**: 在 period 序列中加入 repeat bracket; 拖动 bracket 或 period card 可以改变 repeat 范围。repeat count 使用复刻 Confocal GUI 的 `FluentDoubleSpinBox`: 右侧有蓝色 Plus/Minus 区域、白色上下三角和一个 `.` step 按钮。
- **Preview**: 调用 frontend pulse plot 画未展开 period table 的预览图。默认只画不是 always-off 的 channel, 也可以打开 `Show always-off` 显示全部 channel; 如果 channel 有 display

label, Preview y 轴显示 label。没有 bracket 时, 整体 sequence 持续重复, 左右竖直 bracket 覆盖整段并标记 ∞ 。如果 bracket 覆盖所有 period, 它就是最外层 finite repeat, Preview 标记 xN 。如果 bracket 只包住内部 period, 整体仍然是 ∞ , Preview 会同时画整段 ∞ bracket 和内部 xN bracket, 并用不同颜色区分; 状态栏显示例如 `repeat  $\infty$  + P2-P4 x3`。bracket 的竖线画在真实 start/stop 时间节点上, plot 只通过 xlim 留出左侧显示空间, 并隐藏负时间 tick label。Prepare/Compile 给 FPGA 的 sequence 仍会按 repeat 展开。

- **Save Pulse / Save Figure:** `Save Pulse` 保存 JSON, 默认目录是仓库 `pulses/`, 也可以用 `ZLC_PULSE_DIR` 覆盖; `Save Figure` 在 Preview 顶栏最右侧, 单独保存当前 preview PNG。新建 pulse 的默认名字是 `pulse_YYYYMMDD_HHMMSS`。

无论可见几路, 保存的 `PulseTableState` 都保留完整 `channels` 顺序。隐藏 channel 只是 GUI 视图, 不会改变 bit index。FPGA 端 `mask` 的 bit 仍然按照 `channels[0], channels[1], ...` 排列; 因此 40 路 pin map、server channel list、control PC config 和 GUI state 必须使用同一份 channel 顺序。保存 pulse JSON 时, GUI 会用 pulse name 作为默认文件名; 需要保存预览图时按 `Save Figure`。

仓库 `pulses/camera_imaging_40ch.json` 是按默认 qCMOS 成像时序保存的 40-channel GUI/API preset。硬件 channel 仍然是 `ch00...ch39`; 只有前 4 路显式保存 display label: `ch00=trap`, `ch01=cooling`, `ch02=probe`, `ch03=qcm_trigger`。它使用 100 MHz FPGA 的 10 ns minimal step, 所有 delay 为 0 ns, 默认只显示前 4 路。period timing 对应 `imaging_sequence(exposure=20 ms, load=True, cooling=2 ms, pre_trigger=100 us, trigger_width=20 us)`: cooling 在 0–2 ms 打开; trap 从 0 持续到 22.12 ms; probe 从 2.10 ms 持续到 22.10 ms; qCMOS trigger 在 2.10–2.12 ms 给一个 20 us TTL positive pulse; 最后 20 us 只保留 trap hold, 然后所有输出回到 0。编译到 100 MHz 时 edge tick 为 `[0, 200000, 210000, 212000, 2210000, 2212000]`, mask 为 `[3, 1, 13, 5, 1, 0]`, trigger count 为 1, edge count 为 6, 远小于 1024。

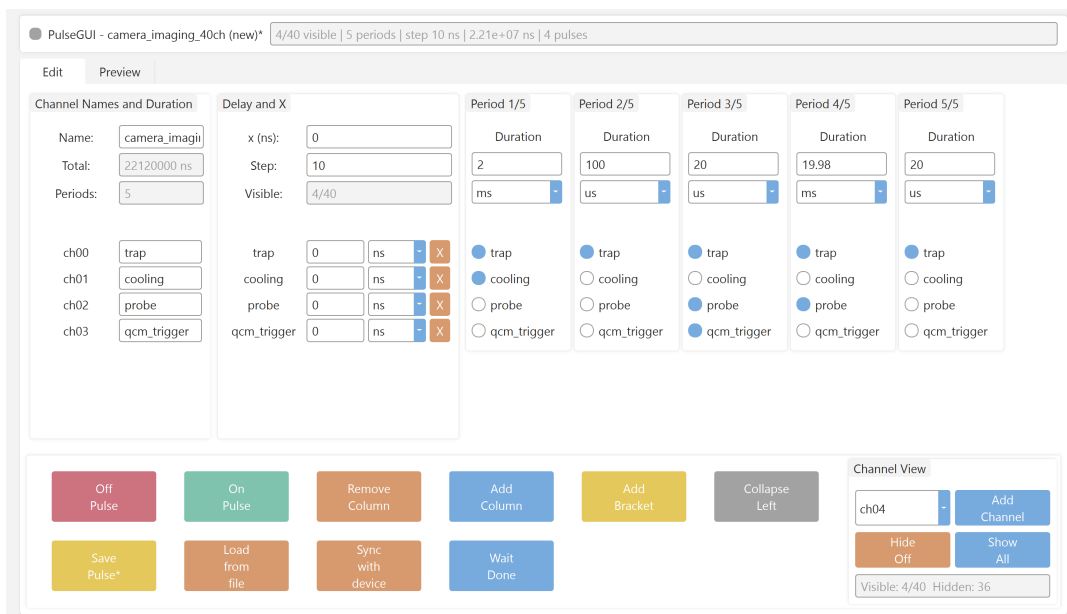


Figure 4.1: 在 Pulse GUI 中打开 `pulses/camera_imaging_40ch.json`。GUI 只显示当前需要编辑的 channel, 但保存和编译仍然保留完整 40-channel 硬件顺序。

推荐工作流

先在 4 路 first-light bitstream 上用 `trap/cooling/probe/qcm_trigger` 做 smoke test; 确认 `prepare → fire → wait_done → safe_state` 后, 再切到 40 路 bitstream。40 路第一次上机时先 `Show All`, 逐路用短 pulse 和示波器确认 pin map; 日常成像时再把不用的 channel 隐藏起来。

4.9 把 FPGA 烧成 pulse-streamer

仓库里已经给出一个 Vivado-ready 入口:

Command line

```
fpga\pulse_streamer\
  zlc_pulse_streamer.v
  zlc_pulse_streamer_top_4ch.v
  zlc_pulse_streamer_4ch.xdc
  create_project_4ch.tcl
  program_fpga_4ch.tcl
  build_4ch_bitstream.bat
  program_4ch_fpga.bat
  zlc_pulse_streamer_top_40ch.v
  zlc_pulse_streamer_40ch.xdc.template
```

`zlc_pulse_streamer.v` 是真正的 runtime pulse-streamer core, `zlc_pulse_streamer_top_4ch.v` 是 first-light top: 它实例化 core 和 Vivado VIO IP, 并把 `out[0:3]` 接到 `trap/cooling/probe/qcm_trigger`。这四个 pin 来自历史 `address_switch` XDC:

4-channel pin map

```
out[0] -> trap          -> PACKAGE_PIN M17
out[1] -> cooling        -> PACKAGE_PIN F15
out[2] -> probe         -> PACKAGE_PIN N15
out[3] -> qcm_trigger   -> PACKAGE_PIN R17
clk      -> PACKAGE_PIN R4
running LED -> PACKAGE_PIN R2
done LED  -> PACKAGE_PIN R3
```

在 Verilog/FPGA 电脑上生成 bitstream:

Command line

```
cd D:\GitHub\Zou_lab_control_v1
.\fpga\pulse_streamer\build_4ch_bitstream.bat
```

这个脚本会调用 `C:\Xilinx \Vivado \2019.2\bin \vivado .bat`, 创建工程、创建 `vio_0` IP、综合、实现并写 bitstream。生成物在:

Command line

```
fpga\pulse_streamer\build\zlc_pulse_streamer_4ch\
  zlc_pulse_streamer_4ch.xpr
  zlc_pulse_streamer_4ch.runs\impl_1\zlc_pulse_streamer_top_4ch.bit
  zlc_pulse_streamer_4ch.runs\impl_1\zlc_pulse_streamer_top_4ch.ltx
```

build Tcl 会检查 `synth_1` 和 `impl_1` 的 run status, 并确认 `.bit/.ltx` 都生成出来; bat

wrapper 会把 Vivado 的 errorlevel 传回 PowerShell。如果 build 没有完成或 probe 文件缺失，不要继续进入 program/smoke/server 步骤。

如果想在 Vivado GUI 里一步步看，也可以打开 `C:\Xilinx \Vivado \2019.2\bin \vivado .bat` 或 Windows Start Menu 里的 Xilinx Vivado 2019.2。若工程还没有生成，先运行 `build_4ch_bitstream.bat`；已有工程后，在 Vivado 里点 `File → Project → Open`，选择 `fpga\pulse_streamer\build \zlc _pulse_streamer_4ch\zlc _pulse_streamer_4ch.xpr`。左侧 Flow Navigator 依次点 `Run Synthesis`、`Run Implementation`、`Generate Bitstream`；每一步完成后底部 Tcl Console/Messages 不应有 error。首次 4-channel build 通常需要 8–20 分钟，电脑较慢或第一次生成 IP cache 时可能到 30 分钟；incremental build 一般 3–10 分钟。

然后烧到 FPGA：

Command line

```
.\fpga\pulse_streamer\program_4ch_fpga.bat
```

烧录脚本会打开 hardware manager、连接第一个 JTAG device、设置 `PROGRAM.FILE`，同时加载 `.ltx` probes，然后执行 `program_hw_devices`。成功后这个 FPGA 就是固定 pulse-streamer bitstream；后续 acquisition 只需要通过 VIO 写 edge table，不需要每次重新综合。

Vivado GUI 的烧录路线是：左侧 `Flow Navigator → Program and Debug → Open Hardware Manager`；顶部绿色栏或 toolbar 点 `Open Target → Auto Connect`；在 Hardware 窗口选中 FPGA device；点 `Program Device`。Bitstream 选择 `...\zlc _pulse_streamer_4ch.runs\impl _1\zlc _pulse_streamer_top_4ch.bit`，Probes file 选择同目录的 `zlc_pulse_streamer_top_4ch.ltx`，然后点 `Program`。JTAG 连接正常时烧录通常 30 秒–2 分钟；第一次打开 Hardware Manager 或驱动枚举较慢时预留 3–5 分钟。

烧录完成后，先不接 qCMOS，使用示波器做 4 路 smoke test：

Command line

```
.\fpga\pulse_streamer\smoke_test_4ch_upload.bat
```

这个脚本会默认把 `ZLC_PS_VIVADO_PROJECT`、`ZLC_PS_VIVADO_BIT` 和 `ZLC_PS_VIVADO_LTX` 指向上面 `build_4ch_bitstream.bat` 生成的 `.xpr/.bit/.ltx`，所以新的 Vivado batch 进程也能重新加载 VIO probes。它会生成一个已知 `RuntimeSequenceProgram`，然后通过当前 `fpga_pulse_streamer` backend 执行 `prepare → fire → wait_done → safe_state`。100 MHz clock 下，预期输出是：

4-channel smoke timing

trap	high 0-10 us
cooling	high 0-3 us
probe	high 2-6 us
qcm_trigger	high 2-3 us

如果这些波形不对，先不要连接相机；检查 `ZLC_PS_VIVADO_BIT`、`ZLC_PS_VIVADO_LTX`、VIO probe 名、XDC pin map 和示波器接线。

first-light 版本的 VIO probe contract 是：

VIO probes

```

probe_out0 zlc_reset      width 1
probe_out1 zlc_start      width 1
probe_out2 zlc_prog_we     width 1
probe_out3 zlc_prog_addr   width 10
probe_out4 zlc_prog_tick   width 32
probe_out5 zlc_prog_mask   width 4
probe_out6 zlc_prog_count  width 11
probe_in0  zlc_running     width 1
probe_in1  zlc_done        width 1

```

Vivado 的 .ltx 里可能把 probe 暴露成语义 net 名, 例如 `zlc_reset` 或 `zlc_vio_i/zlc_reset`; 也可能只暴露成 VIO 端口名, 例如 `probe_out0`. server 生成的 Tcl 会按两套别名查找: `zlc_reset` → `probe_out0`, `zlc_start` → `probe_out1`, `zlc_prog_we` → `probe_out2`, `zlc_prog_addr` → `probe_out3`, `zlc_prog_tick` → `probe_out4`, `zlc_prog_mask` → `probe_out5`, `zlc_prog_count` → `probe_out6`, `zlc_running` → `probe_in0`, `zlc_done` → `probe_in1`. 如果匹配多个 probe, 会打印当前 VIO 上的 probe 列表并报错。正常启动后, `prepare.log` 或 `pulse_streamer_prepare.log` 里应能看到 ZLC pulse-streamer VIO: `zlc_prog_tick=...` 这样的写入记录。

`zlc_pulse_streamer_top_40ch.v` 使用同一个 core, 但是 `prog_mask` 宽度是 40, 输出是 `ch[39:0]`。使用 40 路前, 先把 `zlc_pulse_streamer_40ch.xdc.template` 复制成 `zlc_pulse_streamer_40ch.xdc`, 逐项填入真实 package pin 和连接器含义, 然后运行 `build_40ch_bitstream.bat` 和 `program_40ch_fpga.bat`。如果 `zlc_pulse_streamer_40ch.xdc` 仍然包含 `<PIN_CHxx>` placeholder, 40 路 build 会提前停止。不要在没有确认 40 个物理 pin、电平标准、bank 电压、连接器方向和外部 buffer/level shifter 的情况下烧 40 路 bitstream。40-channel build 比 4-channel VIO 更宽, 通常预留 15–35 分钟; 烧录仍然大约 1–3 分钟。

不使用 Jupyter 时, 在 Verilog/FPGA 电脑的 PowerShell 运行:

Command line

```

cd D:\GitHub\Zou_lab_control_v1
python -m pip install -e .

$env:ZLC_PS_VIVADO_BIN = "C:\Xilinx\Vivado\2019.2\bin\vivado.bat"
$env:ZLC_PS_VIVADO_PROJECT = "$PWD\fpga\pulse_streamer\build\zlc_pulse_st
↪ reamer_4ch\zlc_pulse_streamer_4ch.xpr"
$env:ZLC_PS_VIVADO_BIT =
↪ "$PWD\fpga\pulse_streamer\build\zlc_pulse_streamer_4ch\zlc_pulse_strea
↪ mer_4ch.runs\impl_1\zlc_pulse_streamer_top_4ch.bit"
$env:ZLC_PS_VIVADO_LTX =
↪ "$PWD\fpga\pulse_streamer\build\zlc_pulse_streamer_4ch\zlc_pulse_strea
↪ mer_4ch.runs\impl_1\zlc_pulse_streamer_top_4ch.ltx"
$env:ZLC_PS_VIVADO_PROGRAM_ON_RUN = "0"
$env:ZLC_PS_VIO_FILTER = 'CELL_NAME=~"*vio*"'
$env:ZLC_PS_MAX_EDGES = "1024"
$env:ZLC_PS_TICK_WIDTH = "32"
$env:ZLC_PS_CHANNEL_COUNT = "4"

python -m Zou_lab_control.neutral_atom.devices.sequencer_server `
--host 0.0.0.0 `

```

```

--port 18861 `
--channels trap cooling probe qcm_trigger `
--trigger-channels qcm_trigger `
--clock-hz 100000000 `
--state-dir D:\zlc_sequencer_state `
--prepare-command "python -m
↳ Zou_lab_control.neutral_atom.devices.fpga_pulse_streamer prepare" `
--fire-command "python -m
↳ Zou_lab_control.neutral_atom.devices.fpga_pulse_streamer fire" `
--wait-done-command "python -m
↳ Zou_lab_control.neutral_atom.devices.fpga_pulse_streamer wait_done"
↳ `
--safe-state-command "python -m
↳ Zou_lab_control.neutral_atom.devices.fpga_pulse_streamer safe_state"

```

ZLC_PS_VIVADO_PROGRAM_ON_RUN="1" 会在第一次 run 时烧 bitstream 并加载 probes; 确认成功后可以改成 "0"。如果日后把 VIO upload 换成 UART、JTAG-to-AXI、PCIe 或 Ethernet, 只需要替换 server 的 backend command; control PC notebook 仍然只调用 `RemoteSequencer.prepare/fire/wait_done`。

Backend hooks

```

prepare_callback(program)
fire_callback(program)
wait_done_callback(program, timeout)
safe_state_callback()

```

旧 `legacy_address_switch` backend 仍然保留给 first-light 对比, 但它不是推荐路径。它只能从 sequence 里提取单一 probe width 和 trigger count, 不能表达任意 channel edge table, 也无法保证 qCMOS trigger 与其它 pulse 完全来自同一个 runtime program。

4.10 硬件配置模板

现在有两个 hardware config template:

- `manual_template.json`: QCMOSCamera + ManualSequencer。
- `remote_template.json`: QCMOSCamera + RemoteSequencer。

4.11 device loader 的扩展原则

`load_devices` 保持故意简单: 配置仍然是一个 JSON/dict device graph, 每个节点有 `type` 和 `params`; 依赖用 `"$device:name"` 显式引用; 构造后按 role 检查 `CameraDevice/SequencerDevice/TrapArrayDevice` contract。这个设计比复杂插件系统更适合实验室: 配置文件能直接读懂, 错误会在连接前暴露。

为了让后续硬件扩展不需要反复改 loader 本体, 现在有两个扩展点:

Python

```

na.register_device_class("MySequencer", MySequencer)
na.device_class_registry()

```

`type` 也可以直接写完整 import path, 例如:

JSON

```
{
  "sequencer": {
    "type": "my_lab_devices.fpga.MySequencer",
    "params": {"channels": ["trap", "cooling", "probe", "qcm_trigger"]}
  }
}
```

这样未来加 AOD、microwave、DDS、shutter 或新的 camera adapter 时，只需要实现一个继承对应 base class 的 device，再在 config 中引用它。session、readout、timing 和 notebook 不应该知道这个类内部怎么打开硬件。

真实使用前至少要检查：

- **roi**: qCMOS subarray 的 x、width、y、height 是否对准 trap region。
- **timeout_ms**: 多帧拍摄和 long exposure 是否会超过 timeout。
- **readout_speed**: 与相机模式和 ROI 是否匹配。
- **host/port**: FPGA PC service 地址是否正确。
- **channels/trigger_channels**: Python sequence 里的 channel 名是否和 FPGA output map 对应。

5

配置文件逐项解释

5.1 device graph 的解析方式

`na.load_devices(config)` 接受三种输入: "virtual"、JSON 文件路径、或者 Python dict。JSON/dict 顶层每个 key 是一个 device 名, 例如 `camera`、`sequencer`、`trap_array`。每个 device entry 有两个字段:

Device entry

```
{
  "camera": {
    "type": "QCMOSCamera",
    "params": {
      "config": {
        "exposure": 0.02,
        "roi": [1648, 64, 1144, 64]
      }
    }
  }
}
```

`type` 可以是 registry 里的短名,也可以是完整 Python import path。加载时会实例化 class,然后立刻用 `validate_device_contract` 检查角色。顶层 key 为 `camera` 的对象必须继承 `CameraDevice`; 顶层 key 为 `sequencer` 的对象必须继承 `SequencerDevice`。这比 duck typing 更严格,也更适合实验控制: 错误应该在连接阶段暴露,而不是等到相机已经 arm 以后才发现某个 method 不存在。

本次运行要覆盖某个 device 参数时,使用 `overrides` 或 `na.connect(..., sequencer={...})`,不要在 notebook 里手动读写 JSON:

Python

```
devices = na.load_devices(
    "remote_template",
    overrides={"sequencer": {"host": "192.168.0.20", "port": 18861}},
)
```

5.2 QCMOSCamera 配置

真实 qCMOS template 里的 camera 部分是：

real qCMOS camera

```
"camera": {
  "type": "QCMOSCamera",
  "params": {
    "config": {
      "exposure": 0.02,
      "readout_speed": 1,
      "roi": [1648, 64, 1144, 64],
      "device_index": 0,
      "timeout_ms": 10000
    }
  }
}
```

字段含义：

exposure 默认 exposure，单位秒。真实 acquisition 时 `configure(exposure=...)` 可以改它。

readout_speed DCAM readout speed 参数。不同 qCMOS 模式支持的值可能不同，真实运行前应对照相机 SDK。

roi 当前约定是 `[x, width, y, height]`。ROI 太小会切掉 site；太大则读出慢、timeout 余量小。

device_index DCAM device index。多相机机器上需要确认。

timeout_ms 每次等待 frame ready 的 timeout。多帧 acquisition 中每一帧都会调用 wait；不要把它理解成整个 scan 的总 timeout。

`QCMOSCamera.open()` 只在第一次 acquisition 或显式 `open_devices=True` 时 import dcam 并打开设备。

5.3 ManualSequencer 配置

manual_template 的 sequencer 是：

Manual sequencer

```
"sequencer": {
  "type": "ManualSequencer",
  "params": {
    "channels": ["trap", "probe", "qcm_trigger"],
    "clock_hz": 100000000.0,
    "trigger_channels": ["qcm_trigger"],
    "message": "qCMOS is armed. Start the FPGA trigger sequence now."
  }
}
```

channels 是 sequence validation 和 edge table compilation 的 channel universe。**trigger_channels** 告诉 acquisition 哪些 channel 算 camera trigger。这个名字必须和 `PulseSequence` 里的 channel 一致。如果 FPGA 代码里使用 `camera_trigger` 作为触发通道名，需要把 imaging sequence 改成这个 channel，或把 `trigger_channels` 包含它。

5.4 RemoteSequencer 配置

`remote_template` 的 sequencer 是:

Remote sequencer

```
"sequencer": {
  "type": "RemoteSequencer",
  "params": {
    "host": "192.168.0.20",
    "port": 18861,
    "channels": ["trap", "probe", "qcm_trigger"],
    "clock_hz": 100000000.0,
    "trigger_channels": ["qcm_trigger"],
    "connect_on_init": false
  }
}
```

`connect_on_init=false` 让 session creation 不连接网络。第一次 `prepare`、`fire` 或 `wait_done` 才会 open RPyC connection。调试时可以先:

Python

```
devices = na.load_devices(
    "remote_template",
    overrides={"sequencer": {"host": "192.168.0.20", "port": 18861}},
)
devices.snapshot()
devices.close()
```

这一步不应该要求 FPGA PC 在线。只有真正 `capture` 时才需要 remote service。

6

Acquisition 生命周期

6.1 完整顺序

真实 qCMOS acquisition 的顺序在 `QCMOSCamera.acquire` 里固定：

Acquire lifecycle

```
frames = positive_int(frames)
open camera if needed

if sequence and sequencer:
    runtime_sequence = sequence_for_frame_count(sequence, frames)
    sequencer.prepare(runtime_sequence)

dcam.buf_alloc(frames)
dcam.cap_start(bSequence=True)

if sequence and sequencer:
    sequencer.fire(runtime_sequence)

while next_frame < frames:
    dcam.wait_capevent_frameready(timeout_ms)
    copy available frames from DCAM buffer

sequencer.wait_done(timeout)
dcam.cap_stop()
dcam.buf_release()
```

这个顺序里最关键的是 camera arm 和 sequencer fire 的相对位置。必须先 `cap_start`，再 `fire`；否则第一个 trigger 可能在 qCMOS ready 前到达，结果就是丢第一帧或者整次 acquisition timeout。

6.2 为什么 prepare 在 camera arm 前

`prepare` 做的是 validation、compile、upload 或生成 Verilog。它可以慢，也可以失败。它必须发生在 camera arm 前，因为 camera arm 之后应尽快 fire，而不是再做可能耗时的编译或网络传输。对于

remote sequencer, `prepare` 可以把 edge table 送到 FPGA PC; `fire` 只发一个 start 命令。

6.3 为什么 wait done 在读完 frame 后

camera frame ready 和 sequencer done 是两个不同概念。camera 可能先读到所有 frame, 但 FPGA service 还没确认 finite sequence 结束; 也可能 sequencer 已经结束, 但 camera readout 还在传输。当前顺序是在 frame 全部读出后调用 `wait_done`, 用它确认 timing backend 没有卡死。未来如果某些 backend 能提供 done interrupt, 也可以更早并行等待, 但用户接口不应该变。

6.4 异常路径

如果 `wait_capevent_frameready` timeout, 或者 `wait_done` 返回 false, acquisition 会尝试 `sequencer.abort()`, 然后停止 camera capture 和 release buffer。正常结束不自动 `set_safe_state`, 因为安全态依赖实验: 有的实验希望每 shot 后所有 TTL low, 有的实验希望 trap/hold channel 保持。这个决策应该属于 sequence 或更高层 safety manager, 不应该隐藏在 camera driver 里。

6.5 每一层该记录什么

建议真实实验 logging 分层记录:

- camera: exposure、ROI、readout speed、frame count、timeout、DCAM error。
- sequencer: sequence id、clock、channels、tick/mask hash、trigger count、state。
- timing: human-readable pulse table 和 generated Verilog/manifest path。
- readout result: raw images path、calibration hash、thresholds、occupancy。

当前 result object 和 `exp.save_status()` 已经能保存一部分; 真实硬件接入后应把 raw image stack 和 manifest 一起存档。

7

Readout 算法细节

7.1 site finding

sitemap calibration 的输入是 image stack。当前实现先对 stack 做 reducer, 例如 mean image, 然后找 bright peaks, 再按 grid ordering 排序。这里有两个风险:

- 如果 long exposure 太短, site peak 不稳定, 排序会错。
- 如果 ROI 或 imaging magnification 改了, 旧 sitemap 不能继续用。

因此真实实验上建议每次大幅改 alignment/ROI 后重新跑 sitemap, 并保存 `average_image` 和 `centers`。不要从 trap array simulator 或 AOD geometry 直接推 camera centers; 那些最多是初值, 不是 calibration。

7.2 ROI counts

threshold 和 detect 都从同一个 ROI 定义取 counts:

ROI counts

```
counts = roi_counts(  
    image,  
    calibration.centers,  
    radius=calibration.roi_radius,  
    reducer=calibration.reducer,  
)
```

`radius` 太小会损失 atom signal; 太大则引入 background 和邻近 site cross-talk。当前 tutorial 用小 ROI 只是为了快速跑通。真实实验上应 sweep ROI radius 或用 PSF model 做更稳的积分。

7.3 threshold 和 fidelity

threshold calibration 的核心是从 count distribution 里找 cut。当前 histogram 图做两件事:

- 给出可拖动 threshold line, 让人能快速调整判断边界。

- 尝试用双峰 Gaussian 拟合估计 bright/dark overlap fidelity。

这个 fidelity 是模型估计，不是 ground truth。真实实验中通常不知道每次 shot 是否真的有 atom，所以不能用 simulator occupancy 直接算 accuracy。更可靠的做法是保存 raw counts、reference images 和 fit parameters，再结合物理实验设计，例如 pushout/recapture 或重复探测，交叉验证 readout error。

7.4 detection-time scan 的参考图像

`detection_time` 会先拍 reference exposure images，然后再扫描短 exposure。reference 的作用是用真实可见的 count distribution 给 readout model 一个锚点，而不是偷看 simulator。短 exposure 下的 fidelity 可能出现爬升、平台和下降：

- 爬升来自 signal-to-noise 增强。
- 平台来自 bright/dark peak 可分辨。
- 下降可能来自 atom loss、heating、background 或 state pumping。

如果曲线异常，先看每个 time 的 raw count histogram，而不是只看 fit 后的 fidelity 曲线。

8

真实机器迁移清单

8.1 阶段 0: 离线 smoke test 只跑 virtual tutorial

如果只是想确认 Python package、frontend 和 result object 正常，跑 virtual tutorial：

Notebook

```
tutorials/neutral_atom_tutorial.ipynb
```

不要用 hardware quickstart 做 offline test，因为 hardware quickstart 会真的连接 qCMOS 和 Verilog/FPGA 电脑。

离线 smoke test 只检查：

- 跑完 `tutorials/neutral_atom_tutorial.ipynb`。
- 确认 capture 图没有 sitemap 圈。
- 确认 sitemap、threshold、detect 和 scan 都能返回 result object。
- 确认 `scan.data_figure.decay()` 能直接在 scan 上调用。

8.2 阶段 1: Verilog/FPGA 电脑启动 server

在 Verilog/FPGA 电脑上打开 `tutorials/neutral_atom_fpga_server.ipynb`，或者运行上一章的 PowerShell 命令。看到 server 打印 `Listening on 0.0.0.0:18861` 后，不要关闭这个 terminal/Jupyter kernel。

8.3 阶段 2: control 电脑 real config dry-run

在控制电脑上先只读配置，不打开相机：

Python

```
devices = na.load_devices(  
    "remote_template",  
    overrides={"sequencer": {"host": "192.168.0.20", "port": 18861}},
```



```
)  
devices.snapshot()  
devices.close()
```

这一步应该不需要 qCMOS driver 和 FPGA PC 在线。如果失败, 说明配置文件或 class registry 有问题。然后打开 [tutorials/neutral_atom_hardware_quickstart.ipynb](#) 继续真实连接。

8.4 阶段 3: manual first-light

用 `manual_template` 拍一张图。检查:

- camera 能 open。
- external trigger 后能得到 frame。
- ROI 中能看到光斑或背景。
- timeout 不会过早发生。

这一步不追求自动 scan, 只追求硬件链路通。

8.5 阶段 4: remote service first-light

在 FPGA PC 开 service, 在 control PC 用 `remote_template`。先跑一张 capture, 再跑小 frames 的 sitemap。这里最容易暴露的是 trigger count、network address、service callback 和 FPGA done signal。

8.6 阶段 5: readout calibration

真实 hardware 能拍图之后, 才跑:

```
Python  
  
sitemap = exp.readout.sitemap(frames=20)  
threshold = exp.readout.thresholds(frames=100)  
shot = exp.readout.detect()
```

这一步的输出 calibration 应保存, 并和当天的 timing manifest、camera ROI 一起记录。

9

控制电脑 Jupyter 操作流程

9.1 检查配置和 API

第一格只做 import 和 API 检查。不要在每个 notebook 里写长路径搜索函数。正确做法是在项目根目录安装一次 editable package:

Command line

```
python -m pip install -e .
```

之后 notebook 使用:

Python

```
import Zou_lab_control.frontend as zf
import Zou_lab_control.neutral_atom as na

zf.notebook_setup()
```

9.2 连接真实 hardware session

控制电脑打开 `tutorials/neutral_atom_hardware_quickstart.ipynb`。确认 Verilog/FPGA 电脑 server 已经启动后, 使用:

Python

```
exp = na.connect(
    "remote_template",
    sequencer={"host": "192.168.0.20", "port": 18861},
    open_devices=True,
)
GRID_SHAPE = (5, 7)
```

这里由 `connect(..., open_devices=True)` 统一打开 device graph。失败时不要继续跑后面的 calibration cell。

9.3 拍 raw image

capture 图只显示 raw camera frame:

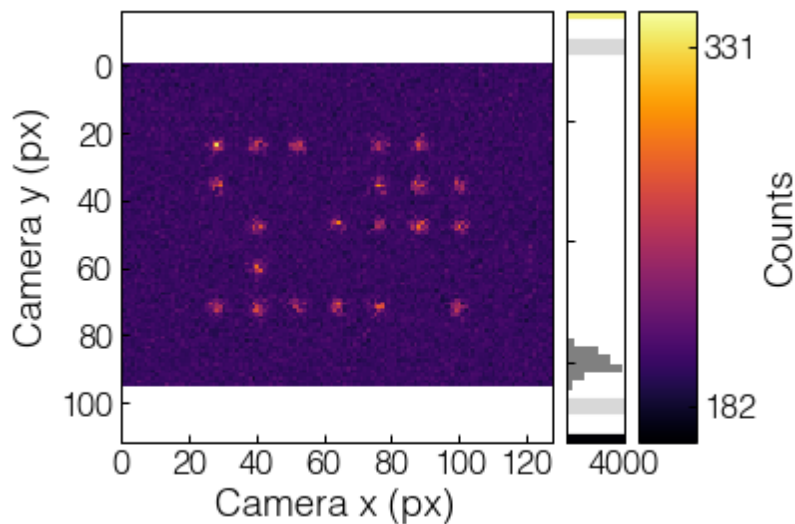


Figure 9.1: capture 只显示 raw frame。没有 sitemap calibration 前，图上不应该出现 site 圈。

对应调用:

Python

```
capture = exp.camera.capture(frames=1, display=True)
capture.images
capture.image
capture.summary()
```

9.4 校准 sitemap

sitemap 使用多张 long exposure/all-sites 图像，输出 camera-space centers:

Python

```
sitemap = exp.readout.sitemap(frames=20, grid_shape=GRID_SHAPE,
    ↪ display=True)
sitemap.centers
sitemap.calibration
exp.readout.current
```

9.5 校准 thresholds

threshold calibration 在 sitemap centers 上取 ROI counts，并为每个 site 建 threshold:

Python

```
threshold = exp.readout.thresholds(frames=80, site=0, display=True)
threshold.counts
threshold.thresholds
```

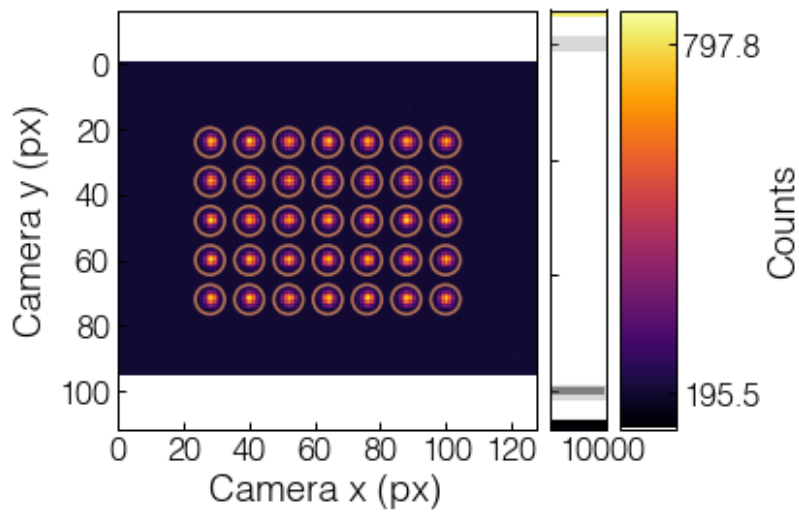


Figure 9.2: sitemap calibration 得到 site centers。圆圈大小由 calibration 的 ROI 半径控制，后续 detect 应使用同一尺度。

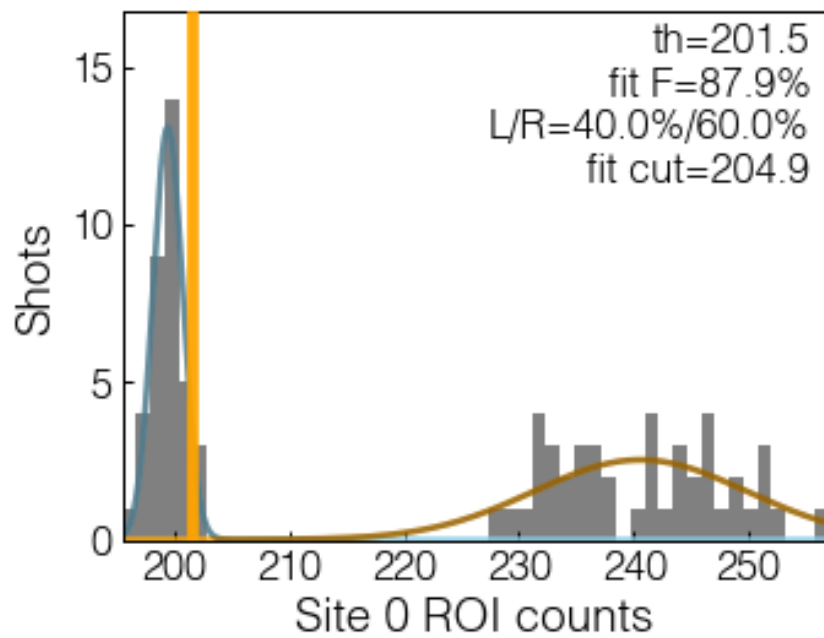


Figure 9.3: threshold histogram。橙色 cut 可拖动；文本显示 cut、左右比例、Gaussian split fidelity 和 fit cut。

```
threshold.plot_site(site=10)
```

9.6 单次 detect

detect 输出 raw image 和 occupancy array:

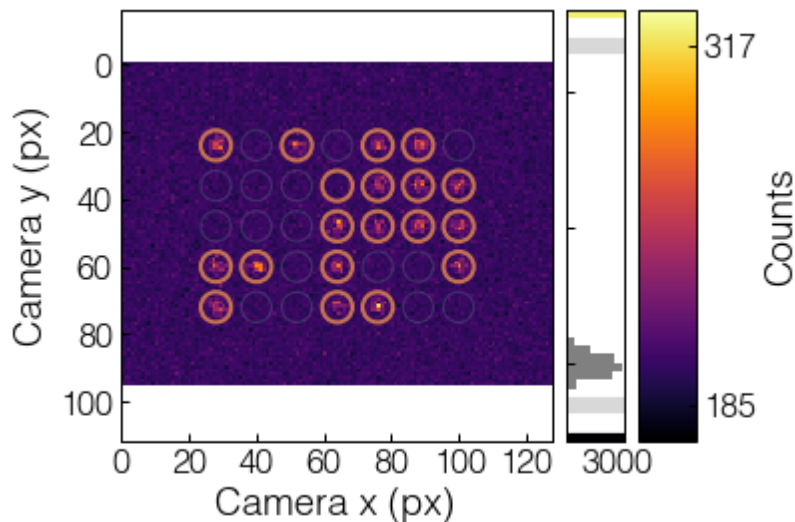


Figure 9.4: detect 图使用浅色背景圈显示 sitemap sites, 只给判定 occupied 的 site 画细橙色圈。

Python

```
shot = exp.readout.detect(display=True)
occupied = shot.occupied
occupied_grid = occupied.reshape(exp.devices.trap_array.grid_shape)
```

注意 `shot.occupied` 是实验后续逻辑需要的 boolean array。图只是 human-facing view。

9.7 scan detection time

detection-time scan 不使用 ground truth。它先拍 reference images, 再对每个 time 拍若干 shots, 从 ROI distribution 估计 threshold 和 Gaussian split fidelity:

Python

```
clock_hz = exp.devices.sequencer.clock_hz
time_ticks = np.linspace(int(round(0.2e-3 * clock_hz)), int(round(8e-3 *
↪ clock_hz)), 60, dtype=int)
times = time_ticks / clock_hz
scan = exp.readout.detection_time(times, shots=20, live=False)
fit_result, popt = scan.data_figure.decay()
```

如果使用默认 `live=True`, cell 会快速返回, 采集在 `frontend.RunSession` owned worker 中继续进行, plot 由 frontend timer 刷新。停止时调用:

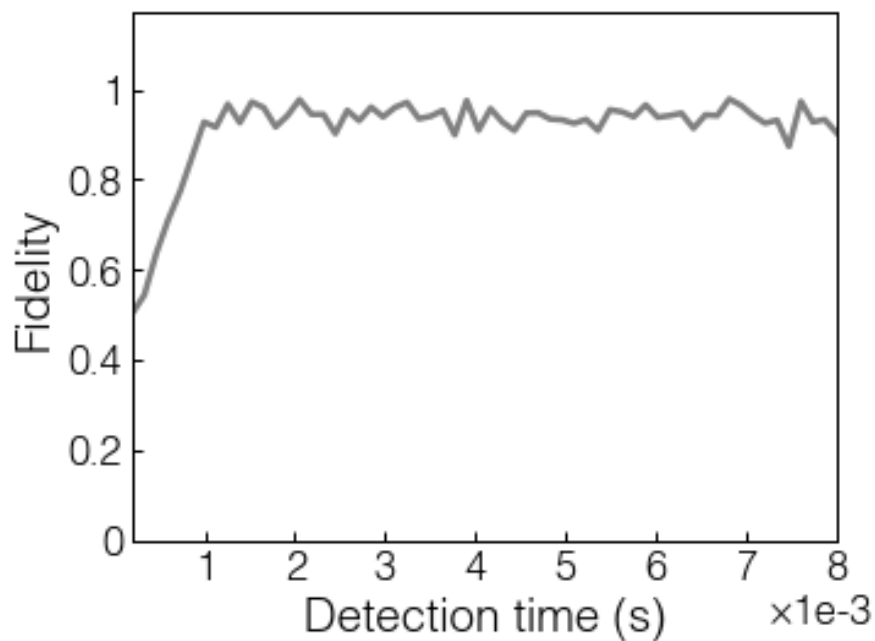


Figure 9.5: detection time vs fidelity scan. 结果对象直接带 `data_figure`, 可以在 scan 上做 decay fit.

Python

```
scan.stop()
```

这不是给用户增加一个额外 controller, 而是让 result object 统一持有 stop 入口。内部的 `measurement`、`plot` 和 `data_figure` 仍然保留, 方便 debug 和 GUI 接管。

10

真实机器上的执行顺序

10.1 first-light 手动触发

第一次连真实 qCMOS 和 FPGA 时, 建议先用 `manual_template`:

Python

```
exp = na.connect("manual_template", open_devices=True)
exp.timing.configure_imaging(exposure=2e-3, load=True)
preflight = exp.timing.preflight()
preflight.raise_if_failed()
capture = exp.camera.capture(display=True)
```

执行到 `capture` 时, qCMOS 会 open、配置 external trigger、alloc buffer、start capture, 然后 `ManualSequencer.fire()` 打印提示。此时手动启动 FPGA trigger。这个模式能检查:

- DCAM driver 是否能打开相机。
- qCMOS external trigger polarity 是否正确。
- ROI/subarray 是否对准 trap image。
- FPGA 输出线和 qCMOS trigger 输入是否接对。

10.2 remote 自动触发

当 FPGA PC 上的 service 准备好后, 用:

Python

```
exp = na.connect(
    "remote_template",
    sequencer={"host": "192.168.0.20", "port": 18861},
    open_devices=True,
)
exp.timing.configure_imaging(exposure=2e-3, load=True)
```

```
capture = exp.camera.capture(display=True)
```

`QCMOSCamera.acquire()` 的顺序是：

1. 根据 `frames` 把 `sequence` 规范成正确 `trigger` 数。
2. `sequencer.prepare(runtime_sequence)`。
3. `dcam.buf_alloc(frames)`。
4. `dcam.cap_start(bSequence=True)`。
5. `sequencer.fire(runtime_sequence)`。
6. 循环等待 `frame ready` 并复制 `image`。
7. `sequencer.wait_done(timeout)`。
8. 停止 `camera capture` 并 `release buffer`。

如果 `camera timeout` 或 `sequencer` 不返回 `done`, `camera acquisition` 会 `abort sequencer`。正常结束时不强制 `set_safe_state()`, 因为未来有些实验可能希望 `trap/holding channel` 在 `shot` 之间保持特定状态; 是否拉低所有输出应该由具体 `sequence` 或安全联锁决定。

10.3 不要把数据采集写成用户线程

长期架构里, 调用者不应该自己起采集线程填 `array`, 也不应该自己管理 `Matplotlib timer`。当前设计是:

- 用户给 `frontend.run(data_x, source)` 一个 `source handle` 或 `callable`。
- `RunSession` 拥有 `acquisition worker`, 按 `data points` 调用 `source`。
- `plot` 用 `frontend timer watch shared array`。
- `result object` 提供 `stop()`, 把采集和 `plot 刷新` 一起停掉。

这样 `Jupyter` 和未来 `PyQt` 都有清晰边界: 硬件采集不阻塞 `UI`, `UI 绘图` 仍留在前端/主线程侧, 实验任务只暴露简单 `result object`。

11

常见问题和排查

11.1 ModuleNotFoundError

如果 notebook 报：

Python

```
ModuleNotFoundError: No module named 'Zou_lab_control'
```

不要在 notebook 顶部写一大段 `sys.path` 搜索。项目根目录执行一次：

Command line

```
python -m pip install -e .
```

然后重启 kernel。教程里的 bootstrap cell 只做简单 import 检查，并在失败时给出这条安装指令。

11.2 capture、sitemap 和 detect 的显示区别

三类图的显示职责不同：

- `exp.camera.capture()` 只显示 raw frame。
- `exp.readout.sitemap()` 显示所有 found sites。
- `exp.readout.detect()` 显示浅色 sitemap background circles 和 occupied circles。

因此，用户通常先用 `capture` 检查相机图像本身，再用 `sitemap` 建立 site calibration，最后用 `detect` 查看带 calibration overlay 的单次占据判断。

11.3 多帧真实采集少帧或 timeout

优先检查：

- `sequence_for_frame_count(sequence, frames)` 是否得到正确 trigger 数。
- `trigger_channels` 是否包含实际 trigger channel 名。

- qCMOS `timeout_ms` 是否足够覆盖 exposure、readout 和 FPGA latency。
- FPGA service 是否真的执行了所有 ticks，而不是只启动一次 single-shot module。

当前代码在 camera arm 前会检查 trigger count；如果 trigger 数明显不匹配，会提前报错而不是让 qCMOS 空等。

11.4 detection-time fidelity 随时间变长反而下降

这不一定是 bug，但要警惕两类原因：

- 真实物理上，exposure 太长会有 atom loss、heating、background 增加或 state pumping，fidelity 确实可能下降。
- 分析上，如果用 simulator ground truth 或 wrong reference，会得到虚假的 fidelity 曲线。

当前实现不使用 simulator truth。它用 long-exposure reference images 和每个 time 的 count distribution 做 threshold/fidelity estimate。真实实验上还应保存 raw counts，检查 bright/dark peaks 是否随 exposure 发生重叠或漂移。

11.5 PyQt 主线程和后台采集

PyQt 绘图必须在主线程是合理约束。当前架构为将来 GUI 留出的边界是：

- device acquisition 可以在 task worker 中运行。
- frontend plot/watch/timer 属于 frontend 层。
- result object 和 shared data array 是 task 与 frontend 的交换边界。
- 不要求用户自己起线程填 array。

将来写 GUI 时，可以把 `RunSession` 的 worker 换成 Qt worker 或实验 scheduler task，把 `ArrayWatcher` 换成 Qt timer，但 `exp.readout.detection_time(...)` 返回 result object 的形状不需要变。

12

下一步扩展计划

12.1 device 层

下一阶段可以逐步补：

- `QCMOSCamera` 的更多 DCAM 参数, 例如 binning、trigger delay、readout mode、metadata。
- 真实 FPGA backend callback, 把 `RuntimeSequenceProgram` 写入 register/FIFO 或 DMA。
- AOD/trap array device, 暴露 waveform upload、site addressing 和 rearrangement command。

12.2 timing 层

timing 层可以继续做：

- 多段 sequence composition, 例如 load、image、pushout、recapture。
- channel delay calibration 和 per-channel polarity。
- Verilog/address-switch 自动生成和 preflight report 对比。
- sequence manifest 保存, 确保每张 camera image 能追溯到当时的 timing。

12.3 readout 层

readout 层可以继续做：

- per-site threshold quality report。
- drift tracking 和 sitemap recenter。
- state-selective detection 和 pushout/dual-image readout。
- fidelity calibration 保存 raw counts、fit parameters 和 reference images。

12.4 frontend 和 GUI 层

frontend 已经提供 array contract、live session、pulse plot、hist threshold、2D map 和 [DataFigure](#) fit。GUI 层应该复用这些对象背后的数据契约，而不是重新发明另一套 plotting state。理想情况下，Jupyter notebook、PyQt panel 和自动实验 scheduler 都能共享：

Shared shape

```
result = exp.readout.some_action(...)
result.plot
result.data_figure
result.summary()
result.stop()
```

这样用户交互保持简单，内部复杂性也有位置可放。